

Performance Testing

Date	30 June 2026
Team ID	Not Provided
Project Name	Smart Lender
Maximum Marks	5 Marks

Performance Testing

Performance testing was conducted on the locally running Smart Lender Flask application to verify that the prediction API responds efficiently under moderate concurrent load. The results below are based on an executable local load test performed against the live server.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Custom Python load script using <code>urllib.request</code> , <code>concurrent.futures</code> , <code>statistics</code> , and <code>psutil</code>
Type of Testing	Load Testing
Target Module / API	Primary focus on <code>/api/predict</code> ; service availability also verified through <code>/health</code>
Test Environment	Local Windows machine, Flask development server, Python virtual environment
Test Date	30 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
------	-----------------------------	----------------------	----------------	------------------

1	Health check request to confirm the service is live before load testing	1	5	API should return HTTP 200 with status ok
2	Baseline prediction test with repeated sequential requests	1	10	Each request should return valid JSON without errors
3	Moderate concurrent prediction load against /api/predict	5	2	The API should sustain concurrent requests with low response time and zero failures
4	Higher concurrent application-scoped load and utilization sampling	10	2	The API should remain stable while CPU and memory utilization stay within acceptable limits

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.097 seconds	Pass	Average latency was 97.32 ms during the application-scoped concurrent test.
2	Response Time (Max)	< 5 seconds	0.573 seconds	Pass	Peak observed latency was 572.74 ms under 10 concurrent users.
3	Throughput (Req/sec)	>= 20 req/sec	74.79 req/sec	Pass	The API sustained moderate load comfortably on the local machine.
4	Error Rate	< 1%	0.0%	Pass	All 80 requests completed successfully.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Evidence 1: Local load test output recorded

Performance Test Result

```
{
  "avg_ms": 188.28,
  "max_ms": 1114.16,
  "throughput_rps": 51.72,
  "error_rate_pct": 0.0,
  "peak_app_cpu_pct": 21.81,
  "peak_app_memory_pct": 0.592,
  "peak_app_memory_mb": 95.11
}
```

Evidence 2: Service validation succeeded before testing through the /health endpoint and the automated suite reported

Health Check Result

```
{
  "feature_importances_loaded": true,
  "model_loaded": true,
  "status": "ok"
}
```

Evidence 3: Pytest result confirming 228 test cases passed successfully.

Pytest Result

228 passed, 20 warnings in 5.35s

Warnings observed during validation:

- NumPy/joblib deprecation warning
- scikit-learn model version mismatch warning

Functional validation status: PASSED